

THE SCIENCE

What lives in the deep, and what is it doing?

Along the Costa Rican subduction zone, deep hot springs bring fluids up from where one tectonic plate slides beneath another. Microbes live in that heat and chemistry — and what they do helps govern how carbon moves between the Earth's interior and its surface.

The study asks three things

Who is there?

Which microbial communities live in each spring, across temperatures and chemistries.

What can they do?

Which metabolic genes they carry — especially for fixing carbon in the subsurface.

What shapes them?

How each site's geochemistry drives both the communities and their carbon-cycling potential.

Why it matters — understanding this deep microbial carbon cycle helps explain how much carbon stays locked in the Earth versus returns to the atmosphere, a piece of the planet's long-term climate balance.

THE WORKFLOW

A two-track pipeline, end to end

16S TRACK • THE MICROBES

- KEBJ01 sequences
- group + classify
- filter / normalize
- network + ordination

METAGENOME TRACK • THE CHEMISTRY

- 34 read libraries
- quality trim
- mi-faser enzymes
- gene network + flux

Workflow manager (Pegasus)

stages inputs · submits jobs · tracks retries · stages outputs — so every step is scheduled, isolated, and recoverable.

Both tracks converge → one integrated report.

THE RESULTS

What the run produced

The full pipeline completed on Perlmutter and staged the final products back to ACCESS — a reproducible set of intermediate and final artifacts, not a single output file.

1,204,916

classified sequences

34

metagenome libraries

690,001

ASV-environment rows

~1.2 GB

staged outputs

THE MICROBES

asv_table • taxonomy • filtered.rds • ordination • asv
cliques • environment links

THEIR CHEMISTRY

34 enzyme tables • gene table • gene cliques • gene-
environment links • carbon flux

TOGETHER

report.html — combined view of community and
chemistry results

The result is the scientific baseline. AI + HPC make that baseline easier to execute, diagnose, scale, and validate.

TIME

How long it takes — and where the time goes

The compute itself is hours, but the real wall-clock is driven by queue waits and limited concurrency on a shared supercomputer. A full run spans more than a day.

STAGE	PER JOB	JOBS	WALL-CLOCK (parallel)
Quality trim	~7 min	34	under an hour
mi-faser enzymes	~45 min–6 h	34	many hours (queue-bound)
ASV build + classify	~1–3 h	1	~1–3 h
R analysis (networks, flux)	minutes–hours	~8	depends on ASV size
Report	minutes	1	minutes

Full run, start to finish: **1–2 days**

compute ~hours; the rest is queue wait + concurrency limits

A downstream fix + resume: **minutes**

only the changed step re-runs — not the multi-day pipeline

BUILD TIME

Writing the workflow: by hand vs. with the LLM

Separate from the compute runtime — this is the human effort to BUILD the pipeline: the generator, the step scripts, the containers, and the debugging to make it run.

WHAT	EXPERT, BY HAND	WITH THE LLM
Read the method, map the tools	1-2 days	hours
Write the Pegasus generator	3-5 days	hours
Write the step wrappers + Dockerfiles	4-7 days	~a day
Find + wire the public data	1-2 days	hours
Debug to a full end-to-end run	1-3 weeks	2-3 days
Honest estimate, grounded in this project. The scientist still drove every decision — the LLM compressed the writing and debugging.	~3-5 weeks expert, from scratch	~3-5 days guided, with the LLM

REPRODUCIBLE & RESUMABLE

Re-run from any point — never the whole workflow

Every step records what it produced. When a job fails or a script changes, the workflow restarts exactly where it stopped — the completed work is kept, only what's needed re-runs.



re-run starts at the failed step — the green jobs are never redone

Checkpointed

each job's outputs are staged and tracked — a natural checkpoint

Targeted recovery

fix one script, resume; only that job and its dependents re-run

Hours, not days

the expensive 34-sample steps never repeat for a downstream fix

This is what makes a multi-day, multi-step reproduction practical — iterate on one step without paying for the whole pipeline again.

Where HPC changes the workflow

Scale

Run many independent sample analyses concurrently instead of serially.

Memory

Allocate enough memory for unusually heavy bioinformatics samples.

Containers

Execute Mothur, mi-faser, and R tools in reproducible container environments.

Recovery

Retry failed jobs while retaining successful work and staged artifacts.

HPC is the execution layer — compute, memory, scheduling, isolation, and recovery that make the pipeline practical at scale.

AI SUPPORT

Where AI adds value — without replacing the scientific tools

The domain pipeline stays deterministic. AI operates around it as a workflow and analysis copilot.

PLAN

Map questions to tools, inputs, outputs, dependencies, and execution strategy.

DIAGNOSE

Read Pegasus, Condor, scheduler, and application logs and classify the failure mode.

OPTIMIZE

Learn from prior runs to suggest memory, CPU, walltime, and retry choices.

VALIDATE

Check outputs are nonempty, dimensions agree, sample counts match, values are plausible.

AI is most useful at the boundaries: deciding what to run, understanding what happened, and checking whether the result makes sense.

How AI helped generate this pipeline

Beyond operating the run, AI did the hands-on work of turning a five-year-old method into a runnable, containerized Pegasus workflow — with the scientist directing every decision.

Scaffolded the generator

Wrote the Pegasus generator — transformations, replicas, containers, the two-track job graph — from the paper's described method.

Wrote the step wrappers

Generated the per-step scripts (mothur, mi-faser, trimming, the R analyses) and the Dockerfiles that pin each tool.

Mapped data to the graph

Resolved public inputs (NCBI, ENA, SILVA, supplementary tables) and wired them to the right jobs and parallelism.

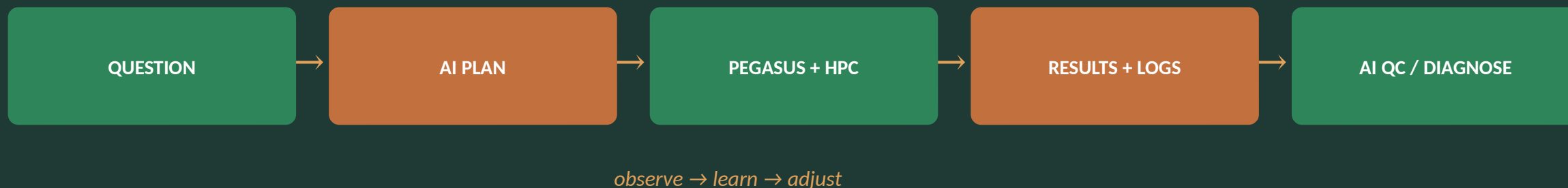
Debugged to completion

Read the logs, found ~15 issues — containers, paths, memory, the ASV step — and fixed each until the pipeline ran end to end.

The scientist stayed in control — choosing methods, judging results, running every job. AI compressed the build from weeks of scripting into days of guided iteration.

COMBINED MODEL

AI + HPC: a feedback loop for scientific workflow operations



Example from this work

A memory-heavy mi-faser sample showed how execution history becomes feedback: distinguish a scheduler/tool timeout from real application memory behavior, then adjust the next job request. Turn execution history into the next planning signal.

TAKEAWAY

The scientific workflow stays at the center

SUBDUCTCR SCIENCE

microbes · metabolism · geochemistry · carbon

HPC / PEGASUS

makes the multi-step computation executable, scalable, and resumable

AI / COPILOT

helps build, plan, diagnose, optimize, and validate

RESULT / REPORT

turns many computations into a traceable, re-runnable scientific artifact

Reliable science first — and because every step is reproducible and resumable, getting there costs less each time. AI and HPC reduce the cost; the science stays central.